

CloudGuardian - Week 3 Live Demo Script

Detect -> Approve -> Remediate -> Verify, end to end, live, in front of your evaluators.

Before you start (do this the night before, not live)

Confirm all of these or the live demo can stall on something boring:

```
terraform -chdir=week3/terraform output # confirm Function App, Event Grid, Logic App all deployed
```

```
curl -s https://<function-app>.azurewebsites.net/api/execute-remediation -X POST -H "x-functions-key: <key>" -d '{}'
```

 # expect a 400 'required' error, NOT a connection failure - proves the endpoint is live

Open, and keep open in browser tabs: (1) the Azure Portal Resource Group overview, (2) Log Analytics -> Logs (with your KQL query pre-typed but not run), (3) the Logic App's Overview > Run History, (4) your approval inbox. Also open a terminal with your Event Grid topic endpoint and key already exported as environment variables so you're not copy-pasting live.

```
export EG_ENDPOINT=$(terraform -chdir=week3/terraform output -raw eventgrid_topic_endpoint) export EG_KEY=$(az eventgrid topic key list --name <topic-name> --resource-group <rg> --query key1 -o tsv)
```

Part 1 - Auto-remediation path (Low/Medium severity, no approval)

#	Action	Command / What happens	Say to evaluator
1	Break it	In the Portal, manually re-enable public blob access on the target Storage Account (simulates drift/a real misconfig).	"I'm going to introduce a real misconfiguration by hand, exactly like an attacker or a careless admin would."
2	Publish the finding	Run the curl command below (Part 1 detail) with severity=Medium.	"This simulates Week 2's pipeline detecting it and publishing to Event Grid - in production this call comes from the CSPM pipeline automatically."
3	Watch it self-heal	Within seconds, refresh the Storage Account's Configuration blade.	"No human touched anything. Medium severity is low-risk and fully reversible, so it's on the auto-remediate path."
4	Show the audit trail	Run the KQL query in the pre-opened Log Analytics tab.	"Every remediation - human-approved or automatic - lands in the same audit trail, queryable with one line of KQL."

Curl command for Part 1, step 2:

```
curl -X POST "$EG_ENDPOINT/api/events" \ -H "aeg-sas-key: $EG_KEY" -H "Content-Type: application/json" \ -d '{ "id": "demo-001", "eventType": "CloudGuardian.Finding.Detected", "subject": "demo", "eventTime": "2026-07-11T10:00:00Z", "dataVersion": "1.0", "data": {
```

```
"finding_id": "demo-001", "control_id": "storage_public_access",  
"remediation_type": "storage_public_access", "severity": "Medium",  
"resource_id":  
"/subscriptions/<sub>/resourceGroups/<rg>/providers/Microsoft.Storage/  
storageAccounts/<name>"    }    }']
```

KQL query for step 4:

```
traces | where customDimensions.event_type == "remediation_result" | project  
timestamp, tostring(customDimensions.control_id),  
tostring(customDimensions.outcome), tostring(customDimensions.resource_id) |  
order by timestamp desc | take 10
```

Part 2 - Human approval path (High/Critical severity)

#	Action	Command / What happens	Say to evaluator
1	Break it worse	Re-enable the SQL Database's TDE=Disabled state (or use a High/Critical finding you already have from a real scan).	"This is a High severity finding - encryption at rest on a database with real (simulated) customer data."
2	Publish the finding	Same curl pattern as Part 1, but severity: "High" and control_id: "sql_encryption".	"Watch - this time nothing happens automatically."
3	Show the approval email	Switch to the approval inbox tab; open the new mail from your ACS sender address.	"Event Grid routed this to the Logic App, which handed it to the Function App. The Function sent this email itself, via Azure Communication Services - no Microsoft 365 mailbox involved anywhere. It's a plain Approve / Reject link, not an Outlook Actionable Message card, deliberately - it works from any inbox."
4	Approve it, live	Click the Approve link.	"That link goes straight to the Function App's approval-decision endpoint, carrying a secret token so a stray click - or an email security scanner pre-fetching the link - can't accidentally trigger a real approval."
5	Show it execute	The browser lands on a small confirmation page; refresh the SQL Database's Transparent Data Encryption blade.	"The click itself is what authorized the Function to make the change - not a separate Logic App step. A confirmation email is also on its way to me now."
6	Show the audit trail	Log Analytics KQL: two records - approval_requested (when the email went out) and approval_decision (when I clicked, and what happened).	"Both the decision and the execution are independently logged and queryable, regardless of which email provider delivered the message."

Part 3 - Governance: drift detection (optional, if time allows)

Manually revert one of the two controls you just fixed (e.g. flip Storage public access back on) WITHOUT publishing a new finding. Then either wait for the daily Automation schedule or manually start the runbook from the Portal (Automation Account > Runbooks > Test-RemediationDrift > Start). Show the job output flagging the drift.

SAY: *This is the difference between remediation and governance - a one-time fix with no follow-up is a false sense of security. This runbook catches silent regressions daily, whether they were accidental or deliberate.*

Part 4 - Privacy-preserving LLM pipeline (if your Week 2 LLM integration is wired up)

Run `privacy_llm's explain_finding()` against a finding containing a real UPN/email in a Python shell, screen-shared. Print the sanitized payload BEFORE the call (tokens visible, no real email/GUID), then the final explanation AFTER (real values restored).

SAY: *Nothing that identifies a real person or tenant resource ever leaves our Azure boundary in the API call itself - only these tokens do. This is a literal implementation of the 'virtual tokens mapped to personal data' safeguard named in India's DPDP Rules, 2025.*

If something breaks live

Have a pre-recorded 90-second screen capture of a full successful run (Part 1 + Part 2) as a fallback. Say so plainly if you switch to it: "Let me show you a recording of this same flow from this morning's dry run, and we can come back to live if we have time at the end" - evaluators respect transparency over a stalled terminal.